Supplementary material for *couplr: Optimal Matching with Verifiable Certificates and Column Generation*

Gilles Colling

Contents

1 What this file holds

The article reports summaries of four measurements that are too large to print in full: the solver grid over problem regimes, the edge-generation loop over covariate counts, point clouds, calipers and sizes, the peak memory of every mode and comparator, and the raw runs behind the scaling table. This file carries the definitions the measurements are stated over, the protocol they were taken under, and summary and worst-case views of each one, and names the script that produced each so any of them can be re-measured. It also carries the parts of the software design the article points at: the object model,

the C++ organisation and the dependency footprint, the proof of the sentinel-padding lemma the article states, the glossary behind the solver figure's labels, and the outcome analysis the article points at.

Every cell of every measurement is in the CSV files listed below, which ship with the submission under `data/`. The tables here select from those files; they do not replace them. Each section names the file that carries its cells in full.

```
## run        20260901-131858
## host       Darwin Mac 25.5.0 Darwin Kernel Version 25.5.0: Mon Apr 27 20:41:15
##            PDT 2026; root:xnu-12377.121.6~2/RELEASE_ARM64_T6041 arm64
## commit     3a083b1 17 modified paths
## R version 4.5.3 (2026-03-11)
## aarch64-apple-darwin25.3.0
## couplr         1.7.0
## MatchIt        4.7.2
## optmatch       0.10.8
## microbenchmark 1.5.0
```

Section	Script	Files
Solver selection across regimes	`scripts/bench_regimes.R`	`regime-runs.csv`, `regime-cells.csv`, `regime-results.csv`, `regime-verdict.csv`
The edge-generation loop	`scripts/bench_implicit_grid.R`	`implicit-grid-runs.csv`, `implicit-grid-results.csv`
Peak memory	`scripts/bench_memory.R`	`memory-results.csv`
Representation against solver	`scripts/bench_implicit.R`	`implicit-results.csv`
Scaling	`scripts/bench_scaling.R`	`scaling-runs.csv`, `scaling-results.csv`
The solver figure	`scripts/make-figure.R`	`benchmark-table.csv`

Every script is resume-safe: it writes its rows as it goes and skips a cell it already holds, so a run that is interrupted continues where it stopped. Moving the CSV aside forces a full re-measurement. `paper/run_bench_suite.sh` runs all of them in order on one machine, which is how the numbers here were produced.

The scripts load couplr from a source checkout with `pkgload::load_all()` and write their CSVs under `paper/`, so they run from a clone of the package repository rather than from the submission archive:

```
git clone https://github.com/gcol33/couplr
cd couplr
sh paper/run_bench_suite.sh
```

The scaling, implicit, implicit-grid, regimes and memory scripts also take `--quick`, which runs a reduced grid.

2 What a tolerance costs the conclusion

`verify_assignment()` decides its conditions in exact arithmetic by default, and the article states the bound that applies when a tolerance is used instead. The derivation is here.

A certificate at tolerance ϵ accepts potentials that violate dual feasibility by ϵ on any pair, miss tightness by ϵ on any matched arc, and leave $|v_j| \le \epsilon$ on any unused column. Write n for the rows and m for the columns, with $n \le m$.

Shifting to $u_i - \epsilon$ and $v_j - \epsilon$ restores exact dual feasibility and the sign condition, because every violated inequality was violated by at most ϵ. The shift costs the dual objective $(n+m)\epsilon$, so the optimum is at least $\sum_i u_i + \sum_j v_j - (n+m)\epsilon$.

Tightness within ϵ on the n matched arcs, and $|v_j| \le \epsilon$ on the $m-n$ unused columns, put the matching's own cost at most $m\epsilon$ above that same sum. Its excess over the optimum is therefore at most $(n+2m)\epsilon$.

At the largest shape in the article's benchmark, 16,667 rows against 33,333 columns, and the default $\epsilon = 10^{-9}$, that is 8×10^{-5} in the cost units.

The bound is why the band is not simply set to zero. The sign of a reduced cost evaluated in double arithmetic can itself be wrong: with $C_{ij} = \pm 2^{-60}$, $u_i = 1$ and $v_j = -1$, the first subtraction rounds to -1 and the expression returns zero on a pair whose reduced cost is not zero. In the negative case that is a dual infeasibility a zero-band check would accept. The exact path splits each rounded addition into the sum and the part the rounding lost, giving an expansion of doubles equal to the difference in the reals, and takes the sign of that expansion (Shewchuk 1997). A rounding-error bound decides which pairs the double evaluation already settles, so the expansion runs near tightness rather than over all nm pairs.

3 The integer conversion bit-scaling performs

`method = "gabow_tarjan"` runs on integers, so a real-valued cost matrix is converted before it is solved and the conversion is part of the method.

Finite costs are shifted so the smallest is zero, multiplied by a scale factor and rounded. The factor is set to keep the scaled costs clear of the sentinel that marks a forbidden pair, and to keep the path sums the solver forms inside 64-bit arithmetic. A matrix whose range leaves no usable factor is refused, with the limit named.

A matrix whose finite entries are each within 10^{-9} of an integer enters unscaled, at the integer nearest each entry. Where those entries are integers the conversion is exact and the optimum returned is the optimum of the instance as supplied. Where they are merely near one, each cost moves by at most 10^{-9}, so the matching returned costs at most $2\min(n, m) \times 10^{-9}$ more than that optimum.

Costs that are not integers are solved on the rounded instance. The matching returned costs at most about 10^{-7} of the cost range more than the optimum of the matrix as supplied at $n = 1000$, and 10^{-5} of it at $n = 10{,}000$ on a square problem.

Potentials are divided by the factor and shifted back, so they belong to the original matrix. Whether the matching is optimal for that matrix rather than only for the rounded instance is a question `verify_assignment()` answers, and it is the reason the article reports solver status and certificate separately.

4 The bound the pricer prunes with

Where the metric admits a ball bound, the columns of a lazy cost source are held in a metric tree and the pricing sweep discards a subtree without visiting it.

For a subtree S and row i the test is

$$L(i, S) - u_i - \max_{j \in S} v_j \geq 0,$$

where $L(i, S)$ is a lower bound on $\min_{j \in S} C_{ij}$ read off the node rather than the minimum itself. If the test holds, no column the node carries can price in, and the subtree is discarded.

A node carries two summaries, because a distance and a caliper are stated in different coordinates. The first is a centre and radius in whitened coordinates, which is what $L(i, S)$ reads. The second is an axis-aligned bounding box in the original covariates, because a caliper is a box there and a whitened ball cannot exclude one; whitening is a rotation and a scaling, so it destroys the axis alignment the per-variable window is stated in.

Mahalanobis distance is Euclidean after whitening, so one tree serves it and the Euclidean metrics alike. Its cost is quadratic in the number of covariates, dear enough to pay for the bound at every dimension measured, so it always takes the tree. The metrics whose cost is linear in the covariates take it only in low dimension. Manhattan and Chebyshev carry no ball bound, a covariance with no Cholesky factor has no whitened coordinates, and a custom distance function is opaque; each falls back to reading the pairs, which is the same answer for more work rather than a different answer.

Every bound is lowered by an allowance before it is compared, so a subtree is discarded only where it clears the threshold by more than the bound can be wrong. The allowance has three parts, and each

covers a different arithmetic: a relative part and an absolute one for the tree's own, and a third for the cost source's, which is what a prune is finally read against.

The relative part covers the rounding of the centre distance and the radius, each a sum of squares and a square root, and for Mahalanobis the residual $E = LL^\top - A$ of the factorization the whitening uses. What a direction d suffers from that residual is $|d^\top E d| / d^\top A d$, which the conditioning of A governs: from $A^{-1} = L^{-\top} L^{-1}$ the ratio is at most $\|E\|_F \|L^{-1}\|_F^2$, and that is the form the allowance carries. A residual measured against the size of A alone would understate it by exactly the conditioning, and understate it most where the geometry is hardest.

The absolute part covers the whitened coordinates themselves. A tree forms them once per point and differences them afterwards, where the distance the cost source computes differences the two points first, so any translation the sample shares survives into the tree's arithmetic and cancels there. Whitening is therefore taken relative to the midpoint of the controls' bounding box, which removes the translation the sample shares, and what rounding remains is bounded by the standard error bound on the dot products forming the whitened displacement. No relative allowance can cover that term, since the whitened coordinates are formed before the difference the distance is taken over and their rounding does not shrink as two points approach.

Both parts above are the tree's own arithmetic, and a prune is not read against the tree's number. It is read against the number the cost source returns, and the two are different computations of the same quantity. The tree accumulates $\|L^\top d\|^2$ as a sum of squares, whose terms are non-negative. The source evaluates $d^\top A d$ by row sums,

$$\sum_a d_a \sum_b A_{ab}\, d_b,$$

whose terms are not. A third part of the allowance covers that evaluation.

Write $\gamma_k = k\varepsilon/(1 - k\varepsilon)$ with ε the unit roundoff, and take the standard model in which each operation contributes a factor $1+\delta$ with $|\delta| \le \varepsilon$. The source recomputes its own differences inside the inner loop, so a term of the inner product carries one subtraction and the length-n accumulation, $(1+\delta)(1+\theta_n) = 1+\theta_{n+1}$. The outer sum multiplies by d_a, itself formed by one subtraction, and accumulates over n terms again, so a term of the double sum carries $(1+\delta)(1+\theta_{n+1})(1+\theta_n) = 1+\theta_{2n+2}$. Summing the term-wise bounds,

$$\left|\, \mathrm{fl}(d^\top A d) - d^\top A d \,\right| \;\le\; \gamma_{2n+2}\, |d|^\top |A|\, |d|,$$

with $|d|$ and $|A|$ taken entrywise.

The right-hand side is $|d|^\top |A| |d|$ and not $d^\top A d$, and that is the whole difficulty. Where A carries off-diagonal entries of mixed sign the row sums cancel, and the ratio $|d|^\top |A| |d| \,/\, d^\top A d$ grows without bound as d runs along a direction A is small in. No relative allowance on the distance can hold an error of that shape. The residual term above does not reach it either: E is exactly zero whenever $LL^\top$ reproduces A in the working precision, which is the ordinary case, so the part of the allowance that appears to scale with conditioning usually contributes nothing at all.

A node is bounded by its box. For a column x the node holds and a row q the displacement is $d = q - x$, so componentwise $|d_a| \le m_a := \max\big(|q_a - \ell_a|,\, |q_a - h_a|\big)$ over the box $[\ell, h]$. Every entry of $|A|$ is non-negative, so the bound is monotone in each $|d_a|$ and

$$s(S) \;=\; \gamma_{2n+2}\, m^\top |A|\, m$$

holds for every column the node carries at once.

s is absolute on the squared distance, so it is applied there and the root retaken: the near side becomes $\sqrt{\max(0,\, d_{\mathrm{lo}}^2 - s)}$ and the far side $\sqrt{d_{\mathrm{hi}}^2 + s}$, each operation rounded away from the bound it widens. Every cost-level question the pricer asks is answered from that pair, so no caller can decide against the geometric bound alone.

For a metric whose squared distance is a sum of non-negative terms, the Euclidean family among them, no cancellation arises: the relative error of that sum is at most γ_n and the tree's relative part already carries it. There s is zero and the geometric bound stands unwidened.

The index is worth stating precisely because it is not the tree's. An earlier version of this allowance charged γ_{n+3}, the count that belongs to the tree's sum of squares, for the source's double sum. Random

search over symmetric A with mixed signs and a wide spectrum reaches a realised error of $5.38\varepsilon\,|d|^{\top}|A||d|$ at $n = 2$ and 6.30ε at $n = 3$, against a γ_{n+3} of about 5ε and 6ε: the earlier constant is not merely loose in the worst case but exceeded by instances a search finds. At γ_{2n+2}, 6ε and 8ε here, both sit inside the allowance.

A subtree that does not clear the threshold is descended into, which costs the evaluations the bound would have saved and not the edge, so an allowance set too high loses time where one set too low would lose an edge.

5 What the restricted solve's optimum can cost

Proposition 1 is exact: if no omitted pair has a negative reduced cost, the restricted optimum is optimal for the complete problem. The loop as it runs weakens that on four counts, and they are worth naming separately rather than folded into one term.

Write n for the rows and m for the columns.

- δ, the duality gap the restricted master's own certificate reports. The restricted solution costs at most $\sum_i u_i + \sum_j v_j + \delta$. Every run in the article reported $\delta = 0$.
- ϵ, the pricing threshold. A pair enters on $\bar{C}_{ij} < -\epsilon$, so termination establishes $\bar{C}_{ij} \geq -\epsilon$ over the omitted pairs rather than $\bar{C}_{ij} \geq 0$. Shifting the row potentials to $u_i - \epsilon$ restores dual feasibility over every admissible pair and costs the dual objective $n\epsilon$.
- The sign condition on the column potentials. Dual feasibility for a rectangular problem also asks $v_j \leq 0$ on the columns an assignment may leave unused. The flow master returns potentials satisfying it by construction on the arcs it carries, and `verify_assignment()` checks it rather than assuming it; shifting u alone does not repair a positive v_j.
- η, the allowance the pricing bound carries where a metric tree is used. A subtree is discarded only where its bound clears the threshold by more than η, so the statement termination establishes is $\bar{C}_{ij} \geq -\epsilon$ with the bound's own error already taken out rather than assumed away.
- The two objective sums' own rounding. δ is a difference of the primal and dual objectives, accumulated over n and over $n + m$ terms. Compensated summation buys back the accumulation error and does not remove it, so each sum carries an envelope of its own. With u the unit roundoff, Neumaier's bound is $\left(2u + O(n^2u^2)\right)\sum_k |x_k|$, charged here as $\left(2u + \gamma_n^2\right)\sum_k |x_k|$, which dominates it.

Taken together the complete optimum is at least $\sum_i u_i + \sum_j v_j - n\epsilon$, and the returned matching's excess over it is at most $\delta + n\epsilon$ plus the two envelopes, with the third and fourth counts entering as conditions that must hold rather than as further slack: the sign condition is checked, and the tree allowance is subtracted before any prune fires. Every step of that assembly is rounded away from zero, so what is reported is an upper bound in double arithmetic and not an estimate of one. On a 20×20 instance with costs near 10^6 the envelopes are the whole of it, about 1.2×10^{-9} against an objective of 2.1×10^6.

At $\epsilon = 10^{-9}$ and the largest shape in the article's benchmark, 16,667 rows, $n\epsilon$ is 2×10^{-5} in the cost units.

The certificate returns this quantity as `max_suboptimality`, so a caller reads what the answer can still be beaten by rather than a bare zero gap. It is computed from the floor the pricing scan proves rather than from ϵ itself, which is the same number where a prune supplied part of the proof and smaller where every pair was evaluated, and it carries m times any positive column potential, so a sign condition that fails widens the bound instead of invalidating it. `certified_reduced_cost_floor` reports that floor beside `min_reduced_cost`, and sits below it exactly where pruning was used.

6 The sentinel padding lemma

Tight constraints can leave the matching problem infeasible, and couplr answers it by replacing the forbidden entries with a finite sentinel and re-solving. The article states the lemma that makes a minimum-cost solve of the padded problem return the lexicographic optimum, the largest admissible matching and the cheapest among those of its size, and defers the proof here.

Write $[\ell, h]$ for the range of the admissible costs of the pruned submatrix, k for the smaller of its two dimensions and σ for the sentinel. Every feasible assignment of the padded problem matches all k rows, so it splits into s sentinel edges and $k - s$ real ones and costs $s\sigma + R$ with $R \in [(k-s)\ell, (k-s)h]$.

Lemma 1. *If $\sigma > k(|\ell| + |h|)$, then a padded assignment using more sentinel edges than another costs strictly more.*

Proof. Let M' and M use s' and s sentinel edges with $s' > s$, and write R' and R for their real costs. Then

$$\text{cost}(M') - \text{cost}(M) = (s' - s)\sigma + R' - R \geq \sigma + (k - s')\ell - (k - s)h \geq \sigma - k|\ell| - k|h| > 0. \qquad \square$$

couplr takes $\sigma = (k+1)(|\ell| + |h|) + 1$, which satisfies the hypothesis whatever the signs of ℓ and h and when the admissible costs are all zero, and leaves a margin of $|\ell| + |h| + 1$ between a matching and one using more sentinel edges. `assignment(cardinality = "maximum")` prices its dummy columns with the same quantity, and a maximisation is solved by negation with the sentinel negated alongside it, so the lemma applies to the negated instance. The reported objective is recomputed over the real pairs, so no sentinel enters it.

The lemma's hypothesis is a statement about the numbers actually used, so it is checkable on the doubles in hand and couplr's σ satisfies it by construction. What the lemma does not settle is the solve. A solver working where a double no longer resolves a gap of $|\ell| + |h| + 1$ cannot be relied on to order two assignments the padding separates by that much, so couplr refuses a padded objective above 2^{53} rather than returning an ordering its arithmetic may not support. That threshold is a precondition on the solve and not a proof of it: 2^{53} bounds the integers a double carries exactly, and the padded costs need not be integers. What establishes that a particular padded solve reached its optimum is the certificate `verify_assignment()` returns for the padded matrix, and the lemma carries that verdict across to the lexicographic objective.

7 Solver glossary

The article's solver figure labels its traces with abbreviations short enough to sit inside a panel. The mapping below gives, for each label, the value the `method` argument of `assignment()` and `lap_solve()` takes, the family the benchmark table groups it under, and the regime the solver is written for. The labels and families are read from `benchmark-table.csv`, the same file the figure is drawn from.

Table 1: Plot label, argument value, family and intended regime for every solver the article's figure draws.

Label	`method =`	Family	Meaning
auto	`auto`	Auto	The dispatch rules, timed as a solver of its own so the probing pass is inside its number.
Auction	`auction`	Auction	Bertsekas auction with adaptive epsilon.
Auction-GS	`auction_gs`	Auction	Gauss-Seidel auction variant; spatial structure.
Auction-S	`auction_scaled`	Auction	Epsilon-scaling auction; large dense problems.
Hungarian	`hungarian`	Classical	The classical Hungarian method, O(n^3).
Munkres	`munkres`	Classical	Matrix-form Kuhn-Munkres, O(n^4), kept as a reference implementation.
CSA	`csa`	Cost scaling	Goldberg-Kennedy cost scaling; medium to large problems.
Gabow-T	`gabow_tarjan`	Cost scaling	Gabow-Tarjan bit scaling with complementary slackness; integer costs.
SSAP-B	`ssap_bucket`	Cost scaling	Dial bucket priority queue over augmenting paths; integer costs.
CS-flow	`csflow`	Flow-based	Successive shortest paths with Johnson potentials on the flow model.
Cycle-C	`cycle_cancel`	Flow-based	Cycle cancelling with Karp's minimum-mean cycle.

Label	`method =`	Family	Meaning
Net-S	`network_simplex`	Flow-based	Network simplex with a spanning-tree representation.
Push-R	`push_relabel`	Flow-based	Goldberg-Tarjan cost-scaling push-relabel.
JV	`jv`	JV / Augmenting path	Column reduction and auction-style initialisation; the general-purpose default on a dense problem.
LAPMOD	`lapmod`	JV / Augmenting path	Sparse Jonker-Volgenant variant over an adjacency list; more than half the entries forbidden.
SAP	`sap`	JV / Augmenting path	Shortest augmenting path over the shared flow model; carries sparsity well.
SAP-D	`sap_dense`	JV / Augmenting path	Shortest augmenting path with a linear scan in place of a heap; a dense cost matrix.
Ramshaw	`ramshaw_tarjan`	Rectangular / general	Ramshaw-Tarjan; strongly rectangular problems, where padding to a square would dominate.
Brute-F	`bruteforce`	Special-purpose	Exact enumeration; at most eight rows and eight columns.
HK-01	`hk01`	Special-purpose	Hopcroft-Karp cardinality matching; binary or constant finite costs.

Three further routines are reachable but not drawn in the figure: the bottleneck objective, which minimises the largest pair cost instead of the sum, Murty's ranking procedure for the k cheapest assignments, and Sinkhorn iteration for the entropy-regularised relaxation.

8 From matched pairs to an estimate

The article shows the matched sample and defers the outcome analysis here. The match is rebuilt below so this file stands on its own.

```
library(couplr)
data(hospital_staff)
treated <- transform(hospital_staff$nurses_extended,   id = nurse_id)
control <- transform(hospital_staff$controls_extended, id = nurse_id)
covars  <- c("age", "experience_years", "certification_level")
m <- match_couples(treated, control, vars = covars, auto_scale = TRUE)
```

Taking the hourly rate as the outcome, the paired difference within each matched pair estimates the association between full-time status and pay among nurses comparable on age, experience and certification.

```
matched <- join_matched(m, treated, control)
d  <- matched$hourly_rate_left - matched$hourly_rate_right
tt <- t.test(d)
c(estimate = mean(d), lower = tt$conf.int[1], upper = tt$conf.int[2])
```

```
## estimate    lower    upper
## 3.531350 1.875651 5.187049
```

Full-time nurses earn 3.53 more per hour than their matched part-time counterparts, with a 95 percent interval that excludes zero. As a paired difference on a matched sample this estimates an adjusted association. Reading it as the causal effect of full-time status is what rests on assumptions no matching method can verify: conditional ignorability, positivity, and the stable unit treatment value assumption (SUTVA), under which a unit's outcome depends on its own treatment alone. The matched output can also be passed to an outcome model for regression adjustment; valid estimation and uncertainty quantification still depend on the estimator used.

What matching can offer against the first of those assumptions is a statement about how strong an unmeasured confounder would have to be to overturn the result. `sensitivity_analysis()` implements

Rosenbaum bounds (Rosenbaum 2002) for the Wilcoxon signed-rank statistic on one-to-one matched pairs, reporting upper and lower p-value bounds across a grid of the odds-ratio parameter Γ.

```
sens <- sensitivity_analysis(m, treated, control, outcome_var = "hourly_rate")
sens$results[sens$results$gamma %in% c(1, 1.5, 1.75, 2), ]
```

```
## # A tibble: 4 x 4
##   gamma t_stat   p_upper  p_lower
##   <dbl>  <dbl>     <dbl>    <dbl>
## 1  1    13304. 0.0000187 1.87e- 5
## 2  1.5  13304. 0.0435    1.01e-11
## 3  1.75 13304. 0.207     4.48e-15
## 4  2    13304. 0.481     1.69e-18
```

At $\Gamma = 1$, meaning no hidden bias, the association is significant. It survives an unmeasured confounder that raises one pair member's odds of being full-time by half, and stops being significant at $\Gamma = 1.75$. A reader can judge from that number whether a confounder of that strength is plausible in this setting; the analysis quantifies exposure to hidden bias rather than removing it.

9 Solver selection across problem regimes

`assignment(method = "auto")` chooses a solver from five rules read off one pass over the cost matrix: enumerate an at most 8 by 8 problem, use a cardinality algorithm where the finite costs are constant or binary, use the sparse Jonker-Volgenant variant where more than half the entries are forbidden, use the shortest-augmenting-path solver where there are at least three times as many columns as rows, and otherwise use Jonker-Volgenant. The article's solver figure varies problem size on square dense integer costs, which is the last rule's regime and no other. This grid varies the properties the rules are stated over.

A cell is a cost regime, a forbidden pattern and a shape. Each cell is generated as several independent instances, each instance is timed several times, and every solver in the panel sees the same instance. The timing a cell reports for a solver is the median across its instances, where each instance contributes the median of its own repetitions, so the spread is between problems rather than between repetitions of one problem. A solver that errored or timed out on a cell has no row there.

Table 2: The factors the solver grid crosses. The admissible shares are the medians of the shares the generated cells actually held. Shapes and sizes are in the next table.

Factor	Level	Meaning
cost regime	int_uniform	integer, uniform on 1..10,000
cost regime	dbl_uniform	double, uniform on (0, 1)
cost regime	binary	binary, 0 or 1
cost regime	constant	constant
cost regime	tied	integer, five distinct values
cost regime	heavy_tailed	double, lognormal
cost regime	metric_uniform	euclidean over uniform points in 8 dimensions
cost regime	metric_clustered	euclidean over eight clusters in 8 dimensions
forbidden pattern	block_4	25 percent of pairs admissible, in 4 disconnected components
forbidden pattern	none	every pair admissible, unstructured
forbidden pattern	random_01	1 percent of pairs admissible, unstructured
forbidden pattern	random_05	5 percent of pairs admissible, unstructured
forbidden pattern	random_25	25 percent of pairs admissible, unstructured
forbidden pattern	random_60	60 percent of pairs admissible, unstructured

Table 3: Shapes. The base tier runs the whole panel at a size every solver in it reaches; the large tier repeats the crossing at a size only the scalable solvers reach.

tier	Shape	aspect	Solvers in the panel
base	500x500	1:1	12
base	500x1500	1:3	12
base	500x5000	1:10	12
large	1000x10000	1:10	5
large	1500x1500	1:1	5
large	1500x4500	1:3	5

9.1 What the rules cost

For each cell, the solver `"auto"` selected is timed as a solver of its own, so its number carries the probing pass the rules cost, and it is compared against the fastest named solver in the same cell. Ratio below is the time the dispatched solver took over the time the cell's fastest named solver took, so 1.00x means the rule picked the best available solver and 2.00x means it cost twice the best.

Table 4: Each dispatch rule, over the cells where it fired.

Rule	Cells	Picked the cell's fastest	Median ratio	Worst ratio
default	144	59	1.14x	10.41x
no_cost_scale	45	34	1.00x	2.18x

Table 5: The same comparison collapsed over each factor of the grid in turn.

Factor	Level	Cells	Picked the cell's fastest	Median ratio	Worst ratio
cost regime	binary	27	16	1.01x	2.18x
cost regime	constant	18	18	1.00x	1.56x
cost regime	dbl_uniform	27	13	1.01x	3.15x
cost regime	heavy_tailed	27	13	1.00x	3.06x
cost regime	int_uniform	27	12	1.05x	3.56x
cost regime	metric_clustered	27	15	1.20x	3.90x
cost regime	metric_uniform	18	6	1.30x	3.34x
cost regime	tied	18	0	3.92x	10.41x
forbidden pattern	25%, 4 blocks	24	12	1.16x	4.31x
forbidden pattern	complete	39	26	1.00x	8.87x
forbidden pattern	1%	39	11	1.21x	3.90x
forbidden pattern	5%	24	11	1.52x	9.07x
forbidden pattern	25%	39	21	1.05x	10.41x
forbidden pattern	60%	24	12	1.21x	7.49x
tier	base	144	65	1.21x	10.41x
tier	large	45	28	1.00x	3.06x

Table 6: The 15 cells where the dispatched solver was furthest off the cell's fastest.

Regime	Pattern	Shape	Picked	Best	Auto (ms)	Best (ms)	Ratio
tied	25%	500x500	jv	auction_scaled	62.1	6	10.41x
tied	5%	500x500	jv	auction_scaled	40.3	4.4	9.07x
tied	complete	500x1500	jv	network_simplex	200	22.5	8.87x

Regime	Pattern	Shape	Picked	Best	Auto (ms)	Best (ms)	Ratio
tied	complete	500x500	jv	network_simplex	57.3	6.6	8.64x
tied	60%	500x500	jv	network_simplex	63.4	8.5	7.49x
tied	60%	500x1500	jv	network_simplex	206	38.3	5.39x
tied	25%	500x1500	jv	network_simplex	198	37.6	5.26x
tied	complete	500x5000	jv	network_simplex	682	133	5.13x
tied	25%, 4 blocks	500x500	jv	auction_scaled	17.5	4.1	4.31x
metric_clustered	1%	500x500	jv	auction_scaled	6.6	1.7	3.90x
int_uniform	1%	500x500	jv	auction_scaled	6.5	1.8	3.56x
tied	60%	500x5000	jv	network_simplex	697	197	3.53x
tied	5%	500x5000	jv	csflow	382	112	3.41x
metric_uniform	5%	500x500	jv	csa	8.9	2.7	3.34x
int_uniform	5%	500x500	jv	csa	8	2.4	3.25x

The rule, the solver it picked, the fastest solver and the ratio between them are in `regime-verdict.csv` for all 189 cells.

9.2 Every solver against the fastest in its cell

The table below reduces each solver to its position across the grid. A solver's ratio in a cell is its median time over the median time of the fastest solver in that cell, so the reduction is within-cell and a solver that entered only the easy cells is not thereby favoured. Solvers differ in how many cells they entered, since the large tier runs a reduced panel and a solver that errored or timed out on a cell has no row there.

Table 7: Every solver over the cells it entered, as a ratio to the fastest solver in the same cell.

Solver	Cells entered	Times fastest	Median ratio	Q75 ratio	Worst ratio	Fastest cell (ms)	Slowest cell (ms)
hk01	36	22	1.00x	1.07x	1.8x	1.48	43.7
auto	189	59	1.07x	1.58x	10.4x	1.44	697
jv	189	36	1.51x	5.35x	65.4x	3.79	5.37e+03
hungarian	144	12	1.91x	8.92x	46.0x	3.77	697
lapmod	189	2	2.05x	7.71x	70.7x	4.32	7.95e+03
csflow	144	16	2.55x	6.71x	56.0x	3.16	937
sap	189	5	2.60x	6.22x	64.2x	2.96	6.55e+03
ramshaw_tarjan	189	0	3.02x	8.35x	108.2x	5.16	1.09e+04
sap_dense	144	0	3.13x	14.12x	76.9x	6.73	2.15e+03
csa	109	5	9.21x	101.67x	1288.2x	1.98	6.45e+03
network_simplex	144	9	14.97x	25.08x	69.6x	3.11	791
auction_scaled	137	23	31.43x	110.67x	675.4x	1.7	8.2e+03

The per-cell medians and quartiles behind this table are in `regime-results.csv`, 1803 rows covering 189 cells and 12 solvers, and the individual timed repetitions are in `regime-runs.csv`, 15216 rows.

9.3 Do the solvers agree

Every solver in a cell solves the same instance, so their totals have to agree. Each run below is compared against the median total of its own cell, and a run counts as off that median where the difference exceeds 10^{-6} of it.

522 instances carry 5 to 12 solvers each. 11 of the 12 solvers return the cell median exactly on every run, and the one that does not is below.

Table 8: The solvers whose total differed from the median total of a cell they ran in.

Solver	Runs	Runs off the median	Regimes	Worst relative difference
auction_scaled	1218	9	heavy_tailed	1.23e-05

`auction_scaled` is a scaling method, and it deviates on the lognormal regime alone, so the rounding a scaling step applies over a cost range that wide may be what separates it from the rest. The deviation is not a property of the regime: the remaining solvers return one common total on every cell of it. Whether a particular matching is optimal for the matrix as supplied is what `verify_assignment()` answers.

10 The edge-generation loop away from one cloud

The article's implicit results come from one cloud: eight independent normal covariates with the treated group shifted, one draw per size. This grid sweeps one factor at a time around that base, several seeds deep. Where the dense path can afford the problem, the loop's answer is compared against it unit by unit.

Table 9: The loop by cell. A sweep moves one factor and holds the rest at the base cell, a gaussian cloud of 10,000 units in 8 dimensions with no caliper. Rounds is the range over seeds. Graph is the median share of the complete pair count the candidate set ended up holding, in percent. Distances is the median number of distance evaluations over all rounds as a multiple of the complete pair count. Implicit and lazy are median seconds over seeds and speedup is their median ratio.

Sweep	Level	Seeds	Rounds	Graph %	Distances	Implicit	Lazy	Speedup
caliper	boundary	5	2-2	1.151	0.31x	0.963 s	1.66 s	1.71x
caliper	mid	5	2-2	1.163	0.51x	1.04 s	1.8 s	1.74x
caliper	none	5	2-2	1.170	1.79x	0.906 s	1.76 s	1.94x
cloud	clustered	5	2-2	1.170	0.48x	0.349 s	0.654 s	1.87x
cloud	contested	5	9-10	73.318	6.67x	133 s	426 s	3.19x
cloud	gaussian	5	2-2	1.170	1.79x	0.905 s	1.76 s	1.95x
cloud	heavy_tailed	5	2-2	1.170	1.99x	0.925 s	0.963 s	1.04x
cloud	lattice_ties	5	2-2	1.170	1.78x	1.14 s	1.96 s	1.72x
cloud	shell	5	4-7	1.183	4.96x	2.77 s	14.8 s	5.29x
cloud	shifted	5	7-7	3.086	5.92x	4.94 s	54.3 s	10.83x
dimension	2 dimensions	5	2-4	1.170	0.24x	0.18 s	0.589 s	3.48x
dimension	4 dimensions	5	2-2	1.170	0.69x	0.37 s	1.43 s	3.99x
dimension	8 dimensions	5	2-2	1.170	1.79x	0.902 s	1.76 s	1.95x
dimension	16 dimensions	5	2-2	1.170	2.00x	2.57 s	4.95 s	1.93x
dimension	32 dimensions	5	2-2	1.170	2.00x	9.58 s	21.3 s	2.22x
size	2,000 units	3	2-2	4.951	1.99x	0.053 s	0.065 s	1.23x
size	5,000 units	3	2-2	2.160	1.92x	0.262 s	0.37 s	1.42x
size	10,000 units	3	2-2	1.170	1.79x	0.909 s	1.76 s	1.95x
size	20,000 units	3	2-2	0.630	1.61x	3.14 s	7.69 s	2.44x
size	50,000 units	3	2-3	0.288	1.94x	23.3 s	64.3 s	2.81x

Table 10: The certificate by cell. The duality gap is the residual the loop stopped on. Equals the dense solve compares the pairing unit by unit where the dense path could be run, and reads not run where the problem was too large for it.

Sweep	Level	Worst duality gap	All certified	Equals the dense solve
caliper	boundary	0.00e+00	TRUE	TRUE
caliper	mid	0.00e+00	TRUE	TRUE
caliper	none	0.00e+00	TRUE	TRUE
cloud	clustered	0.00e+00	TRUE	TRUE
cloud	contested	0.00e+00	TRUE	TRUE
cloud	gaussian	0.00e+00	TRUE	TRUE
cloud	heavy_tailed	0.00e+00	TRUE	TRUE
cloud	lattice_ties	0.00e+00	TRUE	TRUE
cloud	shell	0.00e+00	TRUE	TRUE
cloud	shifted	0.00e+00	TRUE	TRUE
dimension	2 dimensions	0.00e+00	TRUE	TRUE
dimension	4 dimensions	0.00e+00	TRUE	TRUE
dimension	8 dimensions	0.00e+00	TRUE	TRUE
dimension	16 dimensions	0.00e+00	TRUE	TRUE
dimension	32 dimensions	0.00e+00	TRUE	TRUE
size	2,000 units	0.00e+00	TRUE	TRUE
size	5,000 units	0.00e+00	TRUE	TRUE
size	10,000 units	0.00e+00	TRUE	TRUE
size	20,000 units	0.00e+00	TRUE	not run
size	50,000 units	3.64e-12	TRUE	not run

The medians above are taken over seeds, and the seed-to-seed spread is what decides whether they carry. The next table reports, for each cell, the widest separation the seeds produced.

Table 11: Across the seeds of a cell: the range of each quantity and the ratio of the slowest run to the fastest. Seed width is the number of columns the first round gave each row.

Sweep	Level	Seeds	Rounds	Seed width	Graph %	Seconds	Slowest / fastest
caliper	boundary	5	2-2	78-78	1.121-1.168	0.893-1.1	1.23x
caliper	mid	5	2-2	78-78	1.125-1.167	0.902-1.08	1.20x
caliper	none	5	2-2	78-78	1.170-1.170	0.901-0.908	1.01x
cloud	clustered	5	2-2	78-78	1.170-1.170	0.331-0.382	1.15x
cloud	contested	5	9-10	78-78	73.287-73.355	132-134	1.01x
cloud	gaussian	5	2-2	78-78	1.170-1.170	0.902-0.911	1.01x
cloud	heavy_tailed	5	2-2	78-78	1.170-1.170	0.921-0.942	1.02x
cloud	lattice_ties	5	2-2	78-78	1.170-1.170	1.14-1.14	1.00x
cloud	shell	5	4-7	78-78	1.172-2.825	2.05-3.53	1.72x
cloud	shifted	5	7-7	78-78	3.062-3.108	4.85-5.06	1.04x
dimension	2 dimensions	5	2-4	78-78	1.170-1.170	0.149-0.204	1.37x
dimension	4 dimensions	5	2-2	78-78	1.170-1.170	0.359-0.371	1.03x
dimension	8 dimensions	5	2-2	78-78	1.170-1.170	0.897-0.91	1.01x
dimension	16 dimensions	5	2-2	78-78	1.170-1.170	2.56-2.58	1.01x
dimension	32 dimensions	5	2-2	78-78	1.170-1.170	9.56-9.58	1.00x
size	2,000 units	3	2-2	66-66	4.951-4.951	0.052-0.053	1.02x
size	5,000 units	3	2-2	72-72	2.160-2.160	0.26-0.262	1.01x
size	10,000 units	3	2-2	78-78	1.170-1.170	0.903-0.91	1.01x
size	20,000 units	3	2-2	84-84	0.630-0.630	3.12-3.16	1.01x
size	50,000 units	3	2-3	96-96	0.288-0.288	16.1-23.3	1.45x

Every cell of the grid finished: 264 runs over 20 cells and 3 memory modes, all with status ok.

Each of the 264 runs behind these tables is a row of `implicit-grid-runs.csv`, carrying its seed, mode, timing, round count, candidate and evaluated edge counts, duality gap and comparison against the dense solve. The per-cell medians are in `implicit-grid-results.csv`.

11 Peak memory

Peak resident set size is a property of a process, so each cell here is a fresh R process that loads what it needs, does one matching and exits, measured from outside by the platform's `time` utility. The baseline row of a size is a process that loaded the same packages and matched nothing, and it is what the increment column is measured against. `dense_matrix` builds the distance matrix and stops, which separates the representation from the solver that runs on it.

Table 12: Peak resident set size by arm and problem size, in megabytes.

Units	Arm	Peak RSS	Over baseline	R heap peak	Seconds	Outcome
2,000	baseline	146	-	80	0.007	ok
2,000	baseline_alt	153	-	87	0.008	ok
2,000	dense	266	120	153	0.168	ok
2,000	dense_matrix	203	57	137	0.07	ok
2,000	implicit	157	12	102	0.073	ok
2,000	lazy	151	6	102	0.067	ok
2,000	MatchIt	534	381	311	2.37	ok
2,000	optmatch	481	328	318	1.88	ok
5,000	baseline	146	-	80	0.007	ok
5,000	baseline_alt	153	-	87	0.008	ok
5,000	dense	610	464	366	0.845	ok
5,000	dense_matrix	226	80	161	0.342	ok
5,000	implicit	175	29	104	0.327	ok
5,000	lazy	154	8	104	0.365	ok
5,000	MatchIt	2437	2284	1474	16.7	ok
5,000	optmatch	2065	1913	1320	13.4	ok
10,000	baseline	146	-	80	0.008	ok
10,000	baseline_alt	153	-	87	0.007	ok
10,000	dense	1429	1283	812	3.13	ok
10,000	dense_matrix	444	298	374	1.21	ok
10,000	implicit	207	60	108	1.16	ok
10,000	lazy	161	15	108	1.6	ok
10,000	MatchIt	8866	8713	6382	74.3	ok
10,000	optmatch	7344	7191	4545	60.7	ok
20,000	baseline	146	-	80	0.008	ok
20,000	baseline_alt	153	-	87	0.008	ok
20,000	dense	6402	6256	3657	10.7	ok
20,000	dense_matrix	1239	1093	1145	4.25	ok
20,000	implicit	274	128	116	4.12	ok
20,000	lazy	168	22	116	6.4	ok
20,000	MatchIt	8794	8641	7121	113	error
20,000	optmatch	27582	27429	19331	279	ok

Table 13: The four-fold matrix estimate against the measurement, in megabytes. Matrix is the bytes the dense cost matrix itself needs at that shape, and package estimate is what estimate_dense_matrix_mb() predicts, which is four times the matrix. It is the matrix, not the solve: memory_mode = auto reads estimate_dense_solve_mb() instead, which estimates the peak of the solve at twelve times the cell bytes. Both are deliberately conservative, since they exist to refuse a solve that will not fit rather than to predict where a peak lands.

Units	Matrix	Measured over baseline	Measured / matrix	Package estimate
2,000	7	57	8.03	28
5,000	44	80	1.81	178
10,000	178	298	1.68	711
20,000	711	1093	1.54	2844

The MatchIt arm at 20,000 units did not finish, with `error: (from optmatch) result would exceed 2^31-1 bytes`. All 32 measured processes are in `memory-results.csv`, which also carries the matrix and estimate columns for every arm.

12 Representation against solver

The implicit mode is a representation and a solver at once: its restricted master is the flow model, and naming a solver under it is an error rather than a setting. Reading its timing as a representation comparison would therefore attribute the flow solver's behaviour to the representation. The table below is the decomposition: down a column, one solver over two representations; across a row, one representation under two solvers.

Table 14: Wall-clock seconds by representation and solver. Dense and lazy differ only in whether the matrix is materialised; dense and dense-flow differ only in the solver; implicit is the mode as a user meets it.

Units	Dense, JV	Lazy, JV	Dense, flow	Implicit
500	21 ms	8 ms	21 ms	11 ms
2,000	161 ms	53 ms	198 ms	59 ms
5,000	901 ms	350 ms	1.16 s	310 ms
10,000	3.17 s	1.59 s	4.15 s	1.15 s
20,000	not run	6.38 s	not run	4.1 s
50,000	not run	61.9 s	not run	21.3 s

The certificate, pair count, candidate edge count and round count of each of these runs are in `implicit-results.csv`.

13 Scaling

The scaling table in the article reports the median across instances. The dispersion behind those medians is below: an instance is an independently generated problem of that size, a repetition is one timed solve of that problem, and every package sees the same instances.

Table 15: Every size and package of the scaling benchmark. Max over min is the spread across all repetitions of that cell, so it carries both the between-instance and the within-instance variation.

Units	Package	Instances	Runs	Median (s)	Min (s)	Max (s)	Max / min	Outcome
500	couplr	5	15	0.02	0.017	0.022	1.29x	ok
500	MatchIt	5	15	0.138	0.135	0.145	1.07x	ok
500	optmatch	5	15	0.116	0.113	0.125	1.11x	ok
2,000	couplr	5	15	0.152	0.14	0.182	1.30x	ok
2,000	MatchIt	5	15	2.267	2.11	2.334	1.11x	ok
2,000	optmatch	5	15	1.746	1.633	1.887	1.16x	ok
5,000	couplr	5	10	0.788	0.7695	0.825	1.07x	ok
5,000	MatchIt	5	10	16.24	15.98	16.85	1.05x	ok
5,000	optmatch	5	10	12.77	12.35	13.2	1.07x	ok
10,000	couplr	3	6	3.151	3.048	3.294	1.08x	ok
10,000	MatchIt	3	6	75.31	73.81	75.63	1.02x	ok
10,000	optmatch	3	6	60.45	59.14	60.8	1.03x	ok
20,000	couplr	2	2	11.56	10.61	12.52	1.18x	ok
20,000	MatchIt	0	0	-	-	-	-	error
20,000	optmatch	2	2	286.4	280.3	292.6	1.04x	ok
50,000	couplr	1	1	75.7	75.7	75.7	1.00x	ok
50,000	MatchIt	0	0	-	-	-	-	timeout
50,000	optmatch	0	0	-	-	-	-	timeout

Table 16: The runs that did not return a time.

Units	Package	Runs	Outcome
20,000	MatchIt	2	error: (from optmatch) result would exceed 2^31-1 bytes
50,000	MatchIt	1	timeout
50,000	optmatch	1	timeout

Each of the 147 individual repetitions, with its instance index, seed, seconds and status, is a row of `scaling-runs.csv`. The per-cell quartiles are in `scaling-results.csv`.

14 Software design

The article describes the three layers the package is organised in, the dispatch rules, the memory modes and how the package is verified. This section carries the object model, the C++ organisation and the dependency footprint.

14.1 Object model

The object model is S3 throughout. Matching results carry the class vector `c("matching_result", "couplr_result")`, with the design-specific classes (`full_matching_result`, `cem_result`, `subclass_result`) sharing the `couplr_result` parent, which is what lets `balance_diagnostics()`, `join_matched()`, `match_data()` and the `bal.tab()` methods dispatch on it once rather than being rewritten per design. Results are plain lists of tibbles, so a user who wants something the package does not provide can reach in and take it. `lap_solve()` returns a tibble subclassed as `lap_solve_result` with `source`, `target` and `cost` columns, so an optimum is usable in a plot or a join without an extraction step, while `get_total_cost()` and `get_method_used()` recover the metadata carried in attributes.

14.2 C++ organisation

All nineteen solvers are written from scratch in C++ and exposed through Rcpp, with `src/core` holding the cost-source types and shared utilities with no Rcpp dependency, `src/solvers` one algorithm per file

with a thin `_rcpp.cpp` wrapper each, `src/flow` the flow model, its two solvers and the edge-generation loop, and `src/interface` the conversion between R objects and the internal types. The separation means the solver bodies are ordinary C++ that can be compiled and tested outside R, which is what the C++ test suite does.

Solvers are templated on their cost source rather than written against a concrete matrix. A cost source provides `at(i, j)`, `allowed(i, j)` and `empty()`, and two types satisfy that interface: `CostMatrix`, which owns a dense flat array plus a mask, and `LazyCostMatrix`, which owns the two feature matrices and computes C_{ij} on demand from the requested metric, applying calipers and the distance ceiling in `allowed()`. A handful of hot loops index the dense mask array directly for speed, which the lazy type cannot support; these are guarded by a `supports_raw_mask` trait and an `if constexpr` branch rather than by duplicating the solver. One solver body therefore serves both cost representations, and the pricing oracle of the edge-generation loop needs `at(i, j)` and `allowed(i, j)` and nothing else, so it consumes the same concept and adds no third representation of the costs.

`LazyCostMatrix` also bakes in the transformations that the dense path applies in a separate preparation pass, namely mapping forbidden entries to a large sentinel and negating costs when maximising. Doing this at construction is what allows the untouched solver body to work on either type.

14.3 Dependency footprint

couplr declares Rcpp, tibble, dplyr, rlang, purrr and `methods` in `Imports`, and `Rcpp` alone in `LinkingTo`. The recursive closure of hard dependencies is 17 non-base packages, so a default installation brings 18 packages including couplr itself. The comparable figures on the same day were 18 for optmatch, 8 for MatchIt and 7 for designmatch.

Two choices sit behind that number. First, we use no external solver: all nineteen algorithms are package source, so there is no dependency on an LP or MIP library and no system solver to install. That choice also keeps `LinkingTo` to `Rcpp` alone, which matters because every entry there forces a compile of that package before couplr can be built from source; couplr declares only what its headers actually include. Second, everything that serves a single feature lives in `Suggests` and is loaded at call time. The animation and image-morphing functions, the plotting helpers, the interoperability layer for MatchIt, optmatch and cobalt, and the parallel back end are all optional in this sense, so a user who only wants to match pays for none of them.

15 How the feature comparison was read

Each entry of the feature table in the article was read from the alternatives' own source and documentation rather than from their prose descriptions, against MatchIt 4.7.2 and optmatch 0.10.8. The verifier row was assessed on 2026-08-26 and the rest on 2026-08-09.

Cost matrix entry point. All three accept a cost matrix the user supplies. MatchIt documents its `distance` argument as taking "a matrix of pairwise" distances, and optmatch takes one through `match_on()`. What separates the packages is not whether a matrix is accepted but what a pair match is solved as: couplr solves the rectangular assignment directly, while `optmatch::pairmatch()` and MatchIt's optimal method both route it through `fullmatch()`.

Sparse cost support. optmatch represents a sparse cost through its `InfinitySparseMatrix` class. MatchIt's optimal path reaches the same representation: `MatchIt:::matchit2optimal()` calls `optmatch::as.InfinitySparseMatrix()` on its distance object before solving, and adds antiexact constraints to it as further forbidden entries. The row therefore reads for MatchIt as it does for the solver it calls. couplr exploits sentinel `Inf` entries directly in its SAP and LAPMOD solvers.

Public dual-potential verifier. The row asks whether a package exports node potentials together with a function that checks the optimality conditions against a returned solution. optmatch reports a gap instead: the `exceedances` attribute of a `fullmatch()` result is documented as an upper bound, not necessarily sharp, on how far the returned sum of distances exceeds the least possible sum over feasible solutions. Its node prices travel in an `MCFSolutions` attribute that `fullmatch()`'s documentation does not describe, and the function scoring a primal against them is not exported.

Full matching. Both `optmatch::fullmatch()` and couplr's `full_match()` solve the full matching of Hansen and Klopfer, which admits one-to-many and many-to-one matched sets in the same solution. couplr states it as an edge cover over the admissible pairs: a cheapest cover is inclusion-minimal, a minimal cover is a disjoint union of stars, and such a union covering every unit is a full matching.

References

Shewchuk, Jonathan Richard. 1997. "Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates." *Discrete & Computational Geometry* 18 (3): 305–63. https://doi.org/10.1007/PL00009321.